

Bài 8. Lập trình với giao thức mạng

—

Lập trình socket

Khái niệm socket

Socket là một giao diện lập trình ứng dụng (API) giúp gọi tới các chương trình con của hệ điều hành để khai thác các dịch vụ truyền thông TCP, UDP và IP.

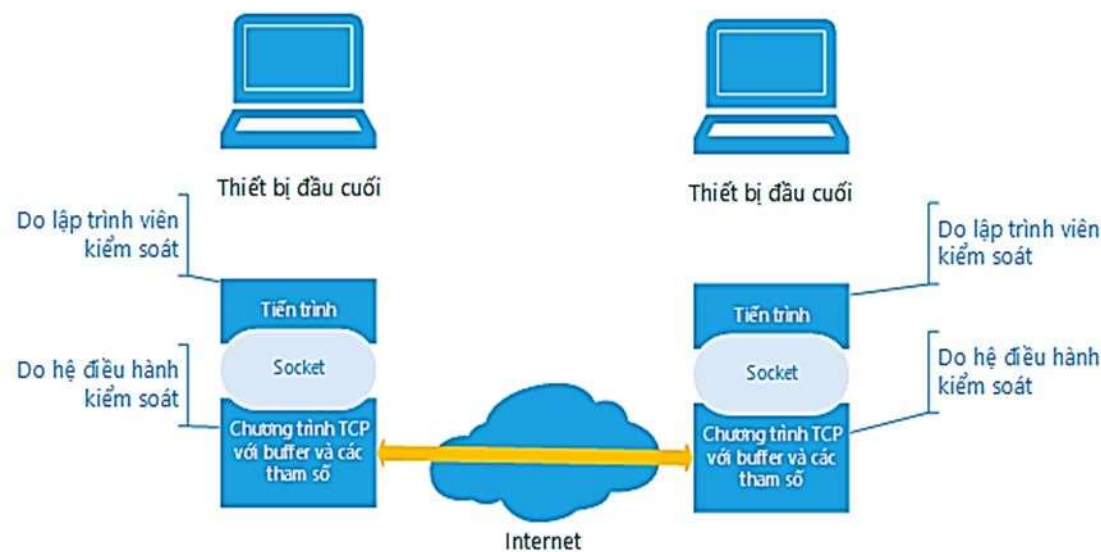
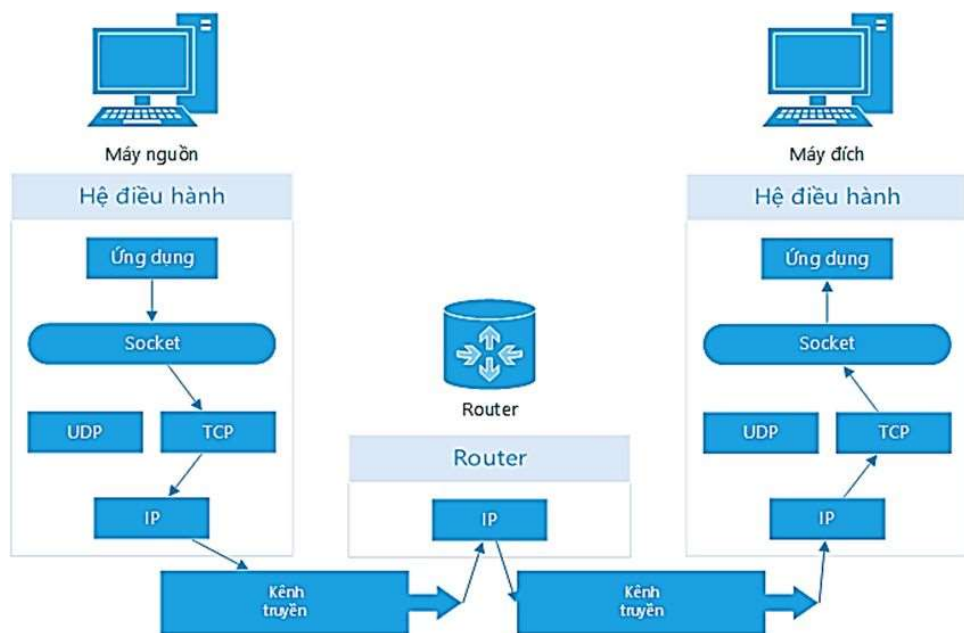
Socket API tiêu chuẩn để lập trình mạng trong Windows gọi là windows socket API (WSA), gọi tắt là winsock.

Phân loại socket API:

- Socket để sử dụng dịch vụ TCP: gọi là TCP socket, socket hướng kết nối (connection-oriented socket), hoặc socket hướng dòng (stream socket).
- Socket để sử dụng dịch vụ UDP: gọi là UDP socket, socket phi liên kết (connectionless socket) hay dgram socket.
- Socket thô (raw socket): là loại socket cho phép gọi thẳng đến dịch vụ IP mà bỏ qua TCP hoặc UDP.

Khi nhìn nhận từ khía cạnh truyền thông (đường đi của thông tin), socket có thể được hình dung như là một cánh cửa ngăn cách giữa chương trình ứng dụng (thuộc toàn quyền của người lập trình) và ngăn xếp giao thức mạng (thuộc quyền quản lý của hệ điều hành)

Nhìn từ mô hình mạng, socket có thể xem như giao diện giữa tầng ứng dụng và tầng giao vận



Lập trình với socket UDP

Bài toán

Viết một ứng dụng dòng lệnh đơn giản với giao diện dòng lệnh (console) theo mô hình chủ / khách sử dụng mô hình truy vấn / phản hồi đáp ứng các yêu cầu sau:

- Client cho phép người dùng nhập một chuỗi ký tự từ bàn phím và gửi chuỗi ký tự cho server;
- Server nhận chuỗi ký tự, chuyển đổi tất cả ký tự thành dạng in hoa và gửi chuỗi kết quả lại cho client;
- Client nhận kết quả và hiển thị lại cho người dùng.

Bước 1. Tạo hai dự án kiểu Console App, một dự án đặt tên là Client, dự án kia đặt tên là Server. Hai dự án đặt trong giải pháp UdpSocket

Bước 2. Thiết lập để đồng thời chạy debug cả hai chương trình.

Bước 3. Viết code cho file Program.cs trong dự án Client (xem dưới đây)

Bước 4. Viết code cho file Program.cs trong dự án Server (xem dưới đây)

Bước 5. Chạy thử ứng dụng bằng phím F5 hoặc tổ hợp Ctrl+F5, sử dụng địa chỉ loopback (127.0.0.1) và số cổng 1308

Nhập bất kỳ chuỗi ký tự nào vào dấu nhắc lệnh của client. Ngay sau đó sẽ nhận được kết quả là chuỗi ký tự đã chuyển thành chữ hoa.

Lưu ý, khi Server hoạt động lần đầu tiên, hệ điều hành sẽ yêu cầu người dùng cấp phép cho Server chiếm dụng cổng 1308. Nếu chạy Server từ một tài khoản bị hạn chế quyền sẽ không thực hiện được yêu cầu này, và Server sẽ bị đóng lại ngay sau đó. Từ lần hoạt động sau sẽ không cần trả lời cấp phép nữa.

```

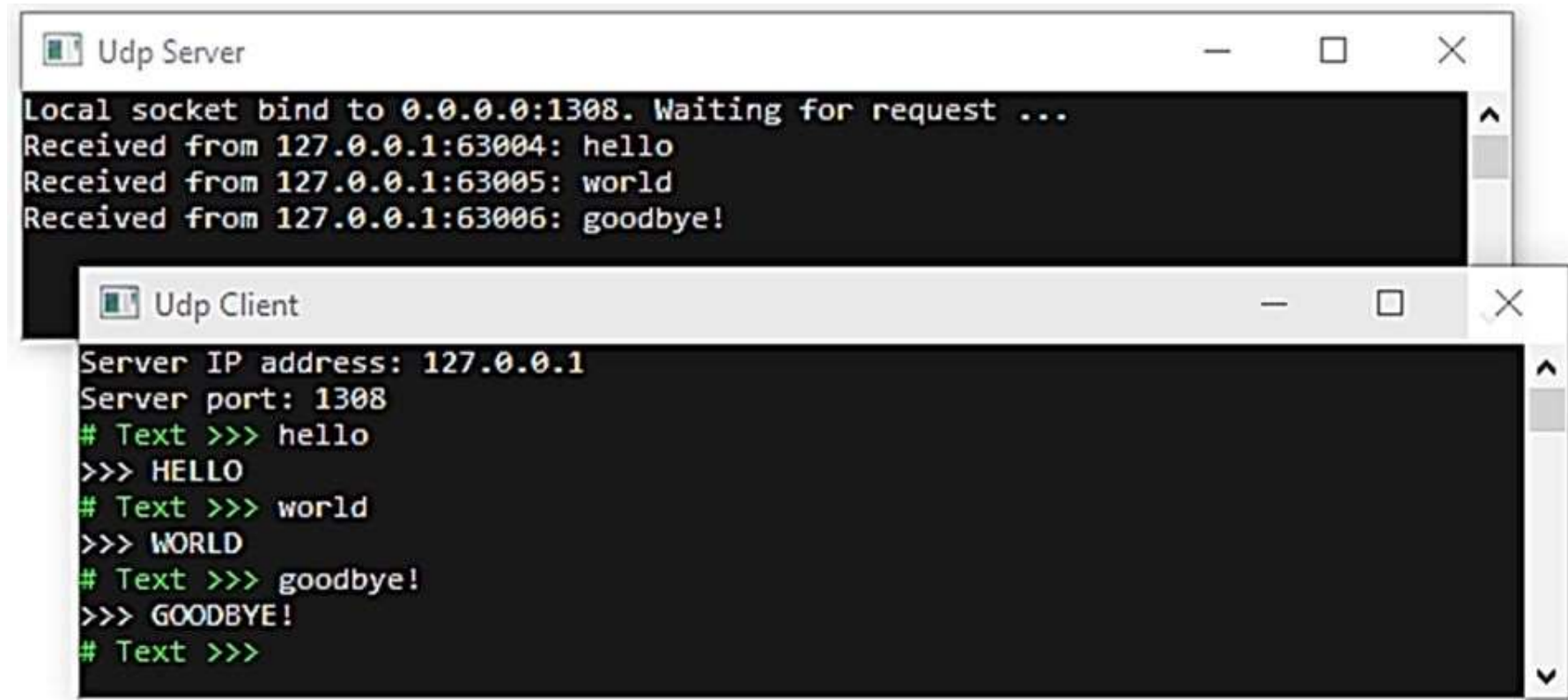
1 using System;
2 using System.Net; // để sử dụng lớp IPAddress, IPEndPoint
3 using System.Net.Sockets; // để sử dụng lớp Socket
4 using System.Text; // để sử dụng lớp Encoding
5 namespace Client;
6 internal class Program {
7     private static void Main(string[] args) {
8         Console.Title = "UDP Client";
9         // yêu cầu người dùng nhập ip của server
10        Console.WriteLine("Server IP address: ");
11        var serverIpStr = Console.ReadLine();
12        // chuyển đổi chuỗi ký tự thành object thuộc kiểu IPAddress
13        var serverIp = IPAddress.Parse(serverIpStr);
14        // yêu cầu người dùng nhập cổng của server
15        Console.WriteLine("Server port: ");
16        var serverPortStr = Console.ReadLine();
17        // chuyển chuỗi ký tự thành biến kiểu int
18        var serverPort = int.Parse(serverPortStr);
19        // đây là "địa chỉ" của tiến trình server trên mạng
20        // mỗi endpoint chứa ip của host và port của tiến trình
21        var serverEndPoint = new IPEndPoint(serverIp, serverPort);
22        var size = 1024; // kích thước của bộ đệm
23        var receiveBuffer = new byte[size]; // mảng byte làm bộ đệm
24        while (true) {
25            // yêu cầu người dùng nhập một chuỗi
26            Console.ForegroundColor = ConsoleColor.Green;
27            Console.WriteLine("# Text >>> ");
28            Console.ResetColor();
29            var text = Console.ReadLine();
30            // khởi tạo object của lớp socket để sử dụng dịch vụ UDP
31            // lưu ý SocketType của UDP là Dgram (datagram)
32            var socket = new Socket(SocketType.Dgram, ProtocolType.UDP);
33            // biến đổi chuỗi thành mảng byte
34            var sendBuffer = Encoding.ASCII.GetBytes(text);
35            // gửi mảng byte trên đến tiến trình server
36            socket.SendTo(sendBuffer, serverEndPoint);
37            // endpoint này chỉ dùng khi nhận dữ liệu
38            IPEndPoint dummyEndPoint = new IPEndPoint(IPAddress.Any, 0);
39            // nhận mảng byte từ dịch vụ UDP và lưu vào bộ đệm
40            // biến dummyEndPoint lưu lại địa chỉ của tiến trình nguồn
41            // tuy nhiên, ở đây chúng ta đã biết tiến trình nguồn là Server
42            // do đó dummyEndPoint không có giá trị sử dụng
43            var length = socket.ReceiveFrom(receiveBuffer, ref dummyEndPoint);
44            // chuyển đổi mảng byte về chuỗi
45            var result = Encoding.ASCII.GetString(receiveBuffer, 0, length);
46            // xóa bộ đệm (để lần sau sử dụng cho yên tâm)
47            Array.Clear(receiveBuffer, 0, size);
48            // đóng socket và giải phóng tài nguyên
49            socket.Close();
50            // in kết quả ra màn hình
51            Console.WriteLine($">>> {result}");
52        }
53    }
54 }

```

```

1 using System;
2 using System.Net;
3 using System.Net.Sockets;
4 using System.Text;
5 namespace Server;
6 internal class Program {
7     private static void Main(string[] args) {
8         Console.Title = "UDP Server";
9         // giá trị Any của IPAddress tương ứng với IP của tất cả các giao diện mạng trên máy
10        var localIp = IPAddress.Any;
11        // tiến trình server sẽ sử dụng cổng 1308
12        var localPort = 1308;
13        // biến này sẽ chứa "địa chỉ" của tiến trình server trên mạng
14        var localEndPoint = new IPEndPoint(localIp, localPort);
15        // yêu cầu hệ điều hành cho phép chiếm dụng cổng 1308
16        // server sẽ nghe trên tất cả các mạng mà máy tính này kết nối tới
17        // chỉ cần gói tin udp đến cổng 1308, tiến trình server sẽ nhận được
18        // một overload khác của hàm tạo Socket
19        // InterNetwork là họ địa chỉ dành cho IPv4
20        var socket = new Socket(AddressFamily.InterNetwork, SocketType.Dgram, ProtocolType.UDP);
21        socket.Bind(localEndPoint);
22        Console.WriteLine($"Local socket bind to {localEndPoint}. Waiting for request ...");
23
24        var size = 1024;
25        var receiveBuffer = new byte[size];
26        while (true) {
27            // biến này sẽ chứa địa chỉ của tiến trình client nào gửi gói tin tới
28            IPEndPoint remoteEndPoint = new IPEndPoint(IPAddress.Any, 0);
29            // khi nhận được gói tin nào sẽ lưu lại địa chỉ của tiến trình client
30            var length = socket.ReceiveFrom(receiveBuffer, ref remoteEndPoint);
31            var text = Encoding.ASCII.GetString(receiveBuffer, 0, length);
32            Console.WriteLine($"Received from {remoteEndPoint}: {text}");
33            // chuyển chuỗi thành dạng in hoa
34            var result = text.ToUpper();
35            var sendBuffer = Encoding.ASCII.GetBytes(result);
36            // gửi kết quả lại cho client
37            socket.SendTo(sendBuffer, remoteEndPoint);
38            Array.Clear(receiveBuffer, 0, size);
39        }
40    }
41 }
42

```



```
Udp Server
Local socket bind to 0.0.0.0:1308. Waiting for request ...
Received from 127.0.0.1:63004: hello
Received from 127.0.0.1:63005: world
Received from 127.0.0.1:63006: goodbye!

Udp Client
Server IP address: 127.0.0.1
Server port: 1308
# Text >>> hello
>>> HELLO
# Text >>> world
>>> WORLD
# Text >>> goodbye!
>>> GOODBYE!
# Text >>>
```

Đề ý rằng mỗi lần Client phát đi một chuỗi ký tự, giá trị cổng của nó lại thay đổi. Trong ví dụ trên, client lần lượt sử dụng các cổng UDP là 63004, 63005, 63006 cho ba lần truy vấn.

Đặc điểm của lập trình với socket UDP:

- (1) Truyền tải dữ liệu qua UDP không sử dụng các bộ nhớ đệm, do đó chương trình đích bắt buộc phải nhận gói tin UDP kịp thời. Nếu không phát lệnh nhận (ReceiveFrom) kịp thời, gói tin UDP sẽ mất đi.
- (2) Chỉ khi nào nhận được dữ liệu (ReceiveFrom), tiến trình đích mới biết được nó đang trao đổi dữ liệu với tiến trình nguồn nào.
- (3) UDP đóng các datagram riêng rẽ cho từng lệnh gửi (SendTo) => Gói tin UDP có thể duy trì ranh giới dữ liệu ứng dụng => Mỗi lệnh gửi (SendTo) phải có một lệnh nhận (ReceiveFrom) tương ứng.

Bài tập: Xây dựng chương trình theo mô hình chủ / khách đáp ứng các yêu cầu sau:

- Client có thể yêu cầu server thực hiện các công việc sau: biến xâu thành chữ hoa, biến xâu thành chữ nhỏ, lấy độ dài xâu.
- Server xử lý xâu ký tự nhận được theo lệnh tương ứng và gửi kết quả lại cho client. Nếu xâu trống hoặc lệnh không phù hợp thì báo lỗi lại cho Client.
- Client và server giao tiếp qua UDP socket.
- Chú ý: Sinh viên tự xây dựng giao thức truyền thông phù hợp và giải thích chi tiết hoạt động của giao thức này.

Lập trình với socket TCP

Bài toán

Viết một ứng dụng dòng lệnh đơn giản với giao diện dòng lệnh (console) theo mô hình chủ / khách sử dụng mô hình truy vấn / phản hồi đáp ứng các yêu cầu sau:

Client cho phép người dùng nhập một chuỗi ký tự từ bàn phím và gửi chuỗi ký tự cho server;

Server nhận chuỗi ký tự, chuyển đổi tất cả ký tự thành dạng in hoa và gửi chuỗi kết quả lại cho client;

Client nhận kết quả và hiển thị lại cho người dùng.

Bước 1. Tạo một Solution trống đặt tên là TcpSocket

Bước 2. Xây dựng lần lượt hai dự án thuộc kiểu Console App trong solution trên với tên gọi lần lượt là Server và Client.

Bước 3. Thiết lập để có thể debug đồng thời cả hai chương trình, trong đó Server sẽ khởi động trước.

Bước 4. Mở file Program.cs của Client và viết code (trang tiếp)

Bước 5. Mở file Program.cs của Server và viết code (trang tiếp)

Bước 6. Dịch và chạy thử hệ thống, kết quả thu được như Hình 5.1.

Khi khởi động chương trình client sẽ yêu cầu nhập địa chỉ IP và số cổng của tiến trình server. Nếu kết nối thành công có thể bắt đầu gửi dữ liệu. Nhập một chuỗi ký tự bất kỳ vào dấu nhắc lệnh. Chuỗi ký tự đó sẽ được chuyển sang server. Server biến chuỗi thành chuỗi in hoa và gửi lại client. Client hiển thị kết quả.

```

3 using System.Net.Sockets;
4 using System.Text;
5 namespace Client;
6 internal class Program {
7     private static void Main(string[] args) {
8         Console.Title = "TCP Client";
9         // yêu cầu người dùng nhập ip của server
10        Console.WriteLine("Server IP address: ");
11        var serverIpStr = Console.ReadLine();
12        // chuyển đổi chuỗi ký tự thành object thuộc kiểu IPAddress
13        var serverIp = IPAddress.Parse(serverIpStr);
14        // yêu cầu người dùng nhập cổng của server
15        Console.WriteLine("Server port: ");
16        var serverPortStr = Console.ReadLine();
17        // chuyển chuỗi ký tự thành biến kiểu int
18        var serverPort = int.Parse(serverPortStr);
19        // đây là "địa chỉ" của tiến trình server trên mạng
20        // mỗi endpoint chứa ip của host và port của tiến trình
21        var serverEndpoint = new IPEndPoint(serverIp, serverPort);
22        var size = 1024; // kích thước của bộ đệm
23        var receiveBuffer = new byte[size]; // mảng byte làm bộ đệm
24        while (true) {
25            // yêu cầu người dùng nhập một chuỗi
26            Console.ForegroundColor = ConsoleColor.Green;
27            Console.WriteLine("# Text >>> ");
28            Console.ResetColor();
29            var text = Console.ReadLine();
30            // khởi tạo object của lớp socket để sử dụng dịch vụ TCP
31            // lưu ý SocketType của TCP là Stream
32            var socket = new Socket(SocketType.Stream, ProtocolType.TCP);
33            // tạo kết nối tới Server
34            socket.Connect(serverEndpoint);
35            // biến đổi chuỗi thành mảng byte
36            var sendBuffer = Encoding.ASCII.GetBytes(text);
37            // gửi mảng byte trên đến tiến trình server
38            socket.Send(sendBuffer);
39            // không tiếp tục gửi dữ liệu nữa
40            socket.Shutdown(SocketShutdown.Send);
41            // nhận mảng byte từ dịch vụ TCP và lưu vào bộ đệm
42            var length = socket.Receive(receiveBuffer);
43            // chuyển đổi mảng byte về chuỗi
44            var result = Encoding.ASCII.GetString(receiveBuffer, 0, length);
45            // xóa bộ đệm (để lần sau sử dụng cho yên tâm)
46            Array.Clear(receiveBuffer, 0, size);
47            // không tiếp tục nhận dữ liệu nữa
48            socket.Shutdown(SocketShutdown.Receive);
49            // đóng socket và giải phóng tài nguyên
50            socket.Close();
51            // in kết quả ra màn hình
52            Console.WriteLine($">>> {result}");
53        }
54    }
55 }

```

```

1 using System;
2 using System.Net;
3 using System.Net.Sockets;
4 using System.Text;
5 namespace Server {
6     internal class Program {
7         private static void Main(string[] args) {
8             Console.Title = "TCP Server";
9             // giá trị Any của IPAddress tương ứng với IP của tất cả các giao diện mạng trên máy
10            var localIp = IPAddress.Any;
11            // tiến trình server sẽ sử dụng cổng tcp 1308
12            var localPort = 1308;
13            // biến này sẽ chứa "địa chỉ" của tiến trình server trên mạng
14            var localEndPoint = new IPEndPoint(localIp, localPort);
15            // tcp sử dụng đồng thời hai socket:
16            // một socket để chờ nghe kết nối, một socket để gửi/nhận dữ liệu
17            // socket listener này chỉ làm nhiệm vụ chờ kết nối từ Client
18            var listener = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.TCP);
19            // yêu cầu hệ điều hành cho phép chiếm dụng cổng tcp 1308
20            // server sẽ nghe trên tất cả các mạng mà máy tính này kết nối tới
21            // chỉ cần gói tin tcp đến cổng 1308, tiến trình server sẽ nhận được
22            listener.Bind(localEndPoint);
23            // bắt đầu lắng nghe chờ các gói tin tcp đến cổng 1308
24            listener.Listen(10);
25            Console.WriteLine($"Local socket bind to {localEndPoint}. Waiting for request ...");
26            var size = 1024;
27            var receiveBuffer = new byte[size];
28            while (true) {
29                // tcp đòi hỏi một socket thứ hai làm nhiệm vụ gửi/nhận dữ liệu
30                // socket này được tạo ra bởi lệnh Accept
31                var socket = listener.Accept();
32                Console.WriteLine($"Accepted connection from {socket.RemoteEndPoint}");
33                // nhận dữ liệu vào buffer
34                var length = socket.Receive(receiveBuffer);
35                // không tiếp tục nhận dữ liệu nữa
36                socket.Shutdown(SocketShutdown.Receive);
37                var text = Encoding.ASCII.GetString(receiveBuffer, 0, length);
38                Console.WriteLine($"Received: {text}");
39                // chuyển chuỗi thành dạng in hoa
40                var result = text.ToUpper();
41                var sendBuffer = Encoding.ASCII.GetBytes(result);
42                // gửi kết quả lại cho client
43                socket.Send(sendBuffer);
44                Console.WriteLine($"Sent: {result}");
45                // không tiếp tục gửi dữ liệu nữa
46                socket.Shutdown(SocketShutdown.Send);
47                // đóng kết nối và giải phóng tài nguyên
48                Console.WriteLine($"Closing connection from {socket.RemoteEndPoint}\r\n");
49                socket.Close();
50                Array.Clear(receiveBuffer, 0, size);
51            }
52        }
53    }
54 }

```

Tcp Server

Local socket bind to 0.0.0.0:1308. Waiting for request ...

Accepted connection from 127.0.0.1:28563

Received: hello world

Sent: HELLO WORLD

Closing connection from 127.0.0.1:28563

Accepted connection from 127.0.0.1:28564

Received: from client

Sent: FROM CLIENT

Closing connection from 127.0.0.1:28564

Accepted connection from 127.0.0.1:28565

Received: goodbye!

Sent: GOODBYE!

Closing connection from 127.0.0.1:28565

Tcp Client

Server IP address: 127.0.0.1

Server port: 1308

Text >>> hello world

>>> HELLO WORLD

Text >>> from client

>>> FROM CLIENT

Text >>> goodbye!

>>> GOODBYE!

Text >>>

Đặc điểm của lập trình với TCP socket:

(1) Xử lý liên kết TCP trong chương trình: duy trì liên kết TCP hay hủy bỏ ngay => Việc lựa chọn mô hình nào phụ thuộc nhiều vào đặc điểm và yêu cầu của phần mềm ứng dụng.

(2) Chương trình TCP sử dụng bộ nhớ đệm cho việc gửi và nhận dữ liệu:

- dữ liệu được chuyển tới một bộ nhớ đệm của TCP (TCP buffer). Theo thời gian, TCP sẽ lấy dữ liệu từ bộ nhớ đệm để đóng gói thành segment. Trong khoảng thời gian chờ ở bộ nhớ đệm, ứng dụng có thể gửi thêm dữ liệu và dữ liệu này sẽ tiếp tục được bổ sung vào bộ nhớ đệm. Như vậy, số segment tạo ra không tương đương với số lệnh Send được gọi.
- TCP trên máy đích cũng sử dụng một bộ nhớ đệm cục bộ của riêng mình. Tất cả dữ liệu đi theo liên kết TCP tới máy đích sẽ được lưu trữ ở bộ nhớ đệm này. Một khi có dữ liệu trong bộ đệm, chương trình có thể đọc được bằng cách gọi phương thức Receive. Lệnh này sẽ di chuyển khối byte từ bộ đệm TCP vào chương trình và xóa bỏ khối byte tương ứng trong bộ đệm. Nếu chương trình đích chưa phát lệnh đọc dữ liệu thì dữ liệu vẫn tiếp tục nằm ở bộ nhớ đệm và không bị mất đi.
- Lệnh Send trên máy nguồn và Receive trên máy đích không đồng bộ nhau. Nhiều lệnh Send gửi dữ liệu đi có thể chỉ cần 1 lệnh Receive để nhận dữ liệu về.

(3) Liên kết và truyền dữ liệu qua TCP giống như một luồng byte liên tục chạy từ buffer của máy nguồn tới buffer của máy đích.

(4) Vì vận chuyển dữ liệu theo một dòng byte liên tục, TCP không đảm bảo duy trì “ranh giới” giữa các thông điệp. Việc phân tách nội dung thông điệp tầng ứng dụng thuộc trách nhiệm của chương trình ứng dụng.

Bài tập:

Xây dựng chương trình theo mô hình chủ / khách đáp ứng các yêu cầu sau:

- Client có thể yêu cầu server thực hiện các phép toán: giai thừa, lũy thừa, logarithm.
- Server thực hiện phép toán và trả kết quả lại cho client. Nếu các tham số không đáp ứng yêu cầu thì báo lỗi lại cho client.
- Client và server giao tiếp qua TCP socket.

Chú ý: Sinh viên tự xây dựng giao thức truyền thông phù hợp và giải thích chi tiết hoạt động của giao thức này.

Phần mềm ứng dụng mạng

Khái niệm phần mềm ứng dụng mạng

Một phần mềm ứng dụng mạng là một hệ thống bao gồm nhiều phần mềm thành phần chạy trên các thiết bị đầu cuối khác nhau và sử dụng mạng để truyền thông tin giữa các thành phần.

Phần mềm mạng là một “hệ thống” mang ý nghĩa rằng ứng dụng mạng bao gồm nhiều thành phần tương đối độc lập nhưng phối hợp hoạt động với nhau trong một thể thống nhất.

Sự phối hợp giữa các thành phần của hệ thống phần mềm mạng có được nhờ:

- Môi trường mạng: cho phép các thành phần trao đổi thông tin;
- Giao thức truyền thông: các quy tắc chi phối quá trình trao đổi thông tin;
- Kiến trúc ứng dụng: sự thỏa thuận về mặt nhiệm vụ giữa các thành phần.

Ví dụ, hệ thống phần mềm web bao gồm hai chương trình có thể chạy độc lập trên các thiết bị đầu cuối khác nhau:

- một chương trình hoạt động trên máy tính người dùng gọi là **trình duyệt web**
- một chương trình chạy trên một máy tính riêng gọi là **chương trình máy chủ web**

Vai trò của các thành phần trong hệ thống web:

- Chương trình máy chủ chứa các trang web tạo sẵn (nhưng không thể hiển thị chúng).
- Chương trình trình duyệt có thể hiển thị các trang web ở dạng đồ họa.

Quy tắc trao đổi thông tin (giao thức) sử dụng trong hệ thống web:

- Trình duyệt và chương trình máy chủ web tương tác với nhau qua mạng máy tính theo mô hình truy vấn - phản hồi (unicast, half-duplex): trình duyệt yêu cầu chương trình máy chủ gửi dữ liệu (các trang web); chương trình máy chủ gửi trang web; trình duyệt hiển thị trang web.
- Trình duyệt và chương trình máy chủ thống nhất sử dụng 2 loại gói tin: truy vấn, phản hồi. Thống nhất cấu trúc 2 loại gói tin dựa trên văn bản.

Mô hình hoạt động phối hợp và cấu trúc thông tin trao đổi này được định nghĩa trong giao thức HTTP.

Lập trình ứng dụng mạng

- Ứng dụng mạng có thể được viết bằng các ngôn ngữ lập trình thông dụng như C/C++, Java, Python, C#, Visual Basic v.v., tương tự như bất kỳ một ứng dụng bình thường.
- Khi lập trình ứng dụng mạng, lập trình viên phải giao tiếp được với ngăn xếp giao thức mạng do hệ điều hành cung cấp để yêu cầu các dịch vụ này thực hiện việc gửi và nhận dữ liệu qua mạng.
- Các hệ điều hành hiện đại cung cấp các dịch vụ truyền thông dưới dạng giao diện lập trình ứng dụng API (Application Programming Interface), gọi là **socket API**.

- Mỗi ứng dụng mạng phải khai thác một trong hai dịch vụ tầng giao vận (dịch vụ truyền thông end-to-end) do hai giao thức tầng giao vận (TCP và UDP) cung cấp.
- Việc khai thác dịch vụ truyền TCP hoặc UDP được thực hiện thông qua khai thác Socket API tương ứng.
- Dịch vụ tầng giao vận sẽ thực hiện các yêu cầu và người lập trình không cần quan tâm đến cách thức chúng thực hiện yêu cầu đó.
- Tùy thuộc vào dịch vụ giao vận được lựa chọn, người lập trình phải tuân thủ theo quy trình hoạt động của giao thức đó. Ví dụ, nếu lựa chọn dịch vụ TCP phải tuân thủ quy tắc hoạt động của TCP, bao gồm: thiết lập liên kết, gửi/nhận dữ liệu, đóng liên kết; Nếu lựa chọn dịch vụ UDP thì có thể trực tiếp gửi/nhận dữ liệu giữa các điểm cuối mà không cần thiết lập liên kết.

Ứng dụng mạng và giao thức tầng ứng dụng

- Khi phát triển ứng dụng mạng luôn phải xây dựng riêng hoặc khai thác một giao thức tầng ứng dụng.
- Giao thức tầng ứng dụng được thực thi thành một module của phần mềm ứng dụng. Đối với một phần mềm đơn giản, giao thức ứng dụng có thể thực thi trực tiếp trong phần mềm. Các giao thức thông dụng thường được thực thi sẵn ở dạng thư viện class để gọi từ phần mềm mà không cần thực thi lại.
- * Lưu ý phân biệt giữa đặc tả của giao thức và thực thi của giao thức. Đặc tả của giao thức là bản thiết kế. Thực thi của giao thức là module phần mềm được viết ra tuân thủ theo thiết kế.

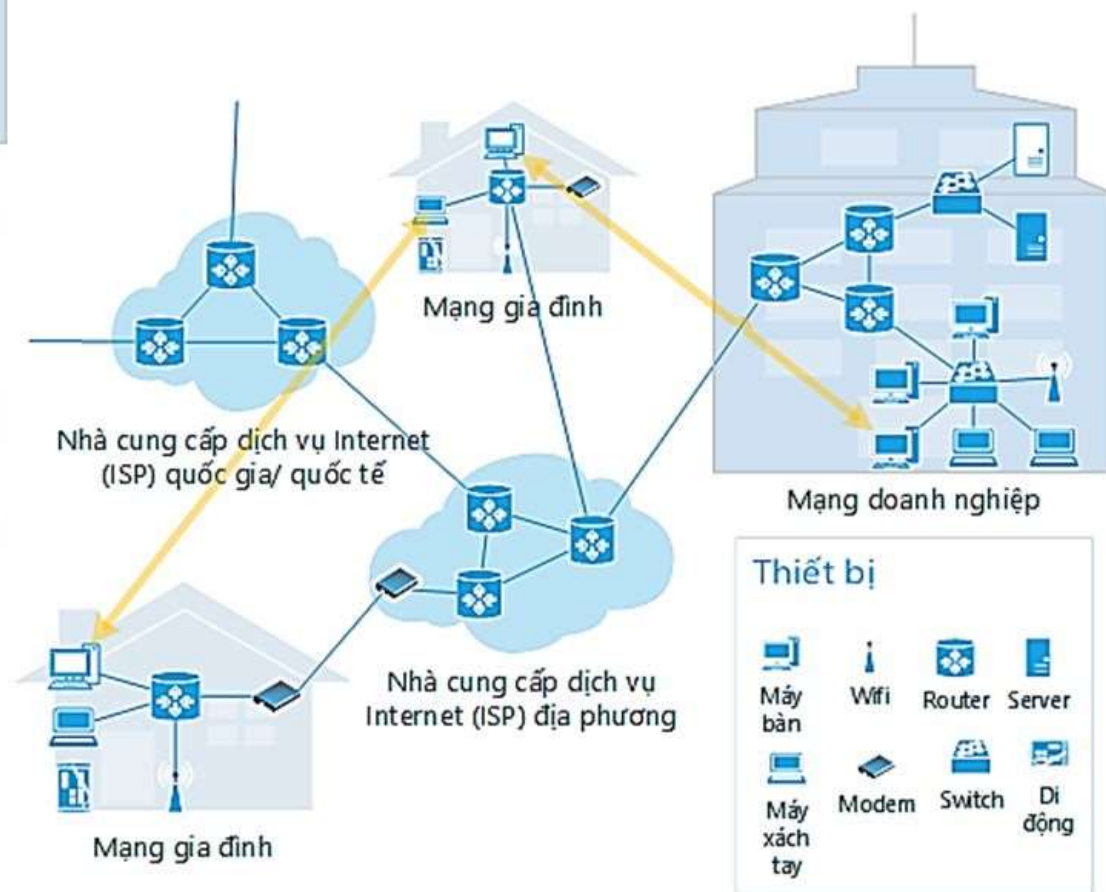
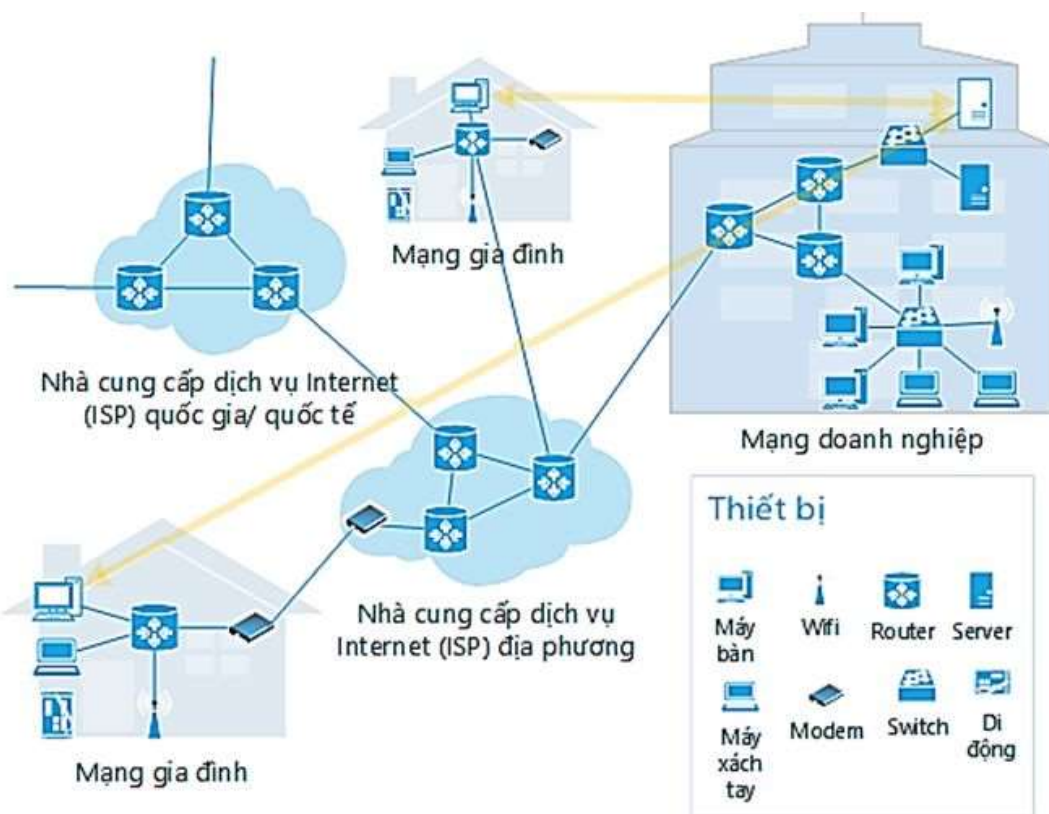
Một giao thức tầng ứng dụng xác định:

- Mô hình trao đổi thông điệp: ví dụ, request/response, publisher/subscriber.
- Các loại thông điệp cần trao đổi: phụ thuộc vào mô hình trao đổi thông điệp. Ví dụ, với mô hình request / response thường xác định thông điệp truy vấn (request) và phản hồi (response).
- Cú pháp của các loại thông điệp: danh sách các trường trong từng loại thông điệp và cách thức tạo ra các trường đó. Giao thức tầng ứng dụng thường sử dụng một trong hai định dạng dữ liệu: chuỗi văn bản (string/text) hoặc chuỗi byte (byte[]).
- Ý nghĩa của các trường: giá trị và vai trò của thông tin trong các trường.
- Các quy tắc xác định thời điểm và cách thức một thành phần gửi thông điệp.
- Loại dịch vụ giao vận nó khai thác: TCP hoặc UDP
- Thỏa thuận về số cổng TCP hoặc UDP

Kiến trúc ứng dụng mạng

Kiến trúc (mô hình) ứng dụng mạng là sự phân chia vai trò nhiệm vụ của các thành phần trong một hệ thống phần mềm ứng dụng mạng, và cách thức các thành phần này tương tác với nhau.

- Mô hình chủ / khách (client / server), trong đó ứng dụng được chia làm hai thành phần: (1) thành phần chuyên phục vụ các yêu cầu gửi đến từ các thành phần khác, gọi là chương trình chủ (server); (2) thành phần đưa ra yêu cầu sử dụng dịch vụ, gọi là chương trình khách (client).
- Mô hình ngang hàng (peer-to-peer, P2P): tất cả các thành phần của hệ thống đều thực hiện các nhiệm vụ giống nhau, không có thành phần phục vụ (server) chuyên biệt, không có thành phần yêu cầu (client) chuyên biệt.



Mô hình trao đổi thông điệp

Mô hình trao đổi thông điệp (messaging pattern), cũng được gọi là mô hình giao tiếp, là một mẫu kiến trúc mô tả cách kết nối và tương tác giữa các thành phần khác nhau của một hệ thống phần mềm phân tán, và xác định cách thức thông điệp được trao đổi trong một hệ thống phần mềm phân tán.

Hai mô hình thường gặp:

- Mô hình Truy vấn (Request) / Phản hồi (Response / Reply)
- Mô hình Xuất bản (Publisher) / Đăng ký (Subscriber)

Mô hình truy vấn / phản hồi có hai thành phần chính:

- Thành phần yêu cầu (requester): Thành phần này gửi yêu cầu và chờ đợi phản hồi tương ứng.
- Thành phần phản hồi (responder): Thành phần này chuyên nhận yêu cầu và phản hồi tương ứng.

Các thành phần yêu cầu không trao đổi thông tin trực tiếp với nhau. Thành phần yêu cầu chỉ trao đổi thông tin với thành phần phản hồi. Do đó, nó tạo ra kiểu trao đổi thông tin 1 - 1 (unicast) truyền hai chiều không đồng thời (half-duplex).

Đây là mô hình trao đổi thông điệp rất phổ biến trong kiến trúc chủ / khách. Trong kiến trúc này, khách thường đóng vai trò yêu cầu, và chủ đóng vai trò phản hồi.

Mô hình truy vấn phản hồi được sử dụng phổ biến trong các giao thức “cổ điển” như HTTP, FTP, các giao thức cho thư điện tử (SMTP, POP).

Mô hình xuất bản/đăng ký bao gồm hai nhóm:

- Người xuất bản (Publisher): Đây là thành phần tạo ra các thông điệp và gửi đi. Mỗi thông điệp được liên kết với một chủ đề cụ thể. Người xuất bản không biết ai sẽ nhận được thông điệp của mình, và trung tâm trung gian sẽ chuyển tiếp thông điệp đến các người đăng ký khi có yêu cầu theo chủ đề tương ứng.
- Người đăng ký (Subscriber): Đây là thành phần nhận thông điệp từ trung tâm trung gian. Người đăng ký chỉ nhận các thông điệp theo chủ đề mà nó đã đăng ký.

Các thông điệp được phân phối đến người đăng ký cùng một lúc và không đồng bộ.

Mô hình xuất bản/đăng ký tạo ra một kiểu trao đổi thông tin 1-nhiều (multicast/broadcast) giữa một người xuất bản và nhiều người đăng ký.

Mô hình xuất bản/đăng ký có hai kiểu triển khai. Kiểu đầu tiên sử dụng một trung tâm trung gian riêng biệt (broker). Kiểu thứ hai sử dụng người xuất bản làm trung tâm trung gian.

Mô hình xuất bản/đăng ký được sử dụng rộng rãi trong các ứng dụng IoT.